

Notebook : Post-Analysis Visualization – Generating Report Figures

This notebook processes the differential expression results (CSVs) generated by the R analysis pipeline (limma/voom).

It generates high-quality, publication-ready visualizations using Python (seaborn and matplotlib) to overcome the limitations of standard R plots.

Objectives:

1. **Smart Volcano Plots:** Distinguish robust signals (Red, $FDR < 0.05$) from biological trends (Orange, Raw $P < 0.01$) to address the limited statistical power.
2. **Top Trend Heatmaps:** Visualize genes strongly associated with the disease (based on raw p-value ranking).
3. **Effect Size Barplots:** Rank top driver genes by $|\log_2 FC|$.
4. **Summary Table:** Automatic generation of the results summary for the report discussion.

Input: .csv files located in Results_R_Analysis/

Output: High-resolution figures in Figures_Finales/

>>> Script R à exécuter dans RStudio <<<

In []:

```
# >>>> SCRIPT R A EXECUTER DANS RStudio
#
=====
===
# SCRIPT: Differential Gene Expression (DGE) & Pathway Enrichment Analysis
# PROJECT: M2 AIDA - Transcriptomic Analysis of the Prefrontal Cortex
#
=====
===

# -----
-----
# 1. ENVIRONMENT SETUP & DATA LOADING
# -----
-----

rm(list = ls())
graphics.off()

cat("=====
\n")
```

```

cat("🚀 STARTING R ANALYSIS PIPELINE: DGE & ENRICHMENT\n")
cat("=====\n")

# --- A. Locate Export Directory ---
input_dir <- "exports"
if (!dir.exists(input_dir) && dir.exists("export")) { input_dir <- "export"
}

if (!dir.exists(input_dir)) {
  if (file.exists("pseudobulk_counts.csv")) {
    input_dir <- "."
  } else {
    stop("CRITICAL ERROR: 'exports' directory not found.")
  }
}

print(paste("📁 Data Directory identified:", input_dir))

# --- B. Load Data ---
file_counts <- file.path(input_dir, "pseudobulk_counts.csv")
file_meta <- file.path(input_dir, "pseudobulk_metadata.csv")

if (!file.exists(file_counts)) stop("ERROR: 'pseudobulk_counts.csv' is
missing.")
if (!file.exists(file_meta)) stop("ERROR: 'pseudobulk_metadata.csv' is
missing.")

counts <- read.csv(file_counts, row.names = 1, check.names = FALSE)
metadata <- read.csv(file_meta, row.names = 1)

cat("✅ Data loaded successfully.\n")

if(!all(colnames(counts) == rownames(metadata))) {
  stop("DATA INTEGRITY ERROR: Count matrix columns do not match metadata
rows.")
}

# --- C. RECODING DISEASE LABELS (CRITICAL FIX) ---
# Mapping long names to short codes for analysis
cat("🔄 Recoding disease labels to standard codes (AD, PD, CTRL)...\n")

metadata$disease[metadata$disease == "dementia || Alzheimer disease"] <-
"AD"
metadata$disease[metadata$disease == "dementia || Parkinson disease"] <-
"PD"
metadata$disease[metadata$disease == "normal"] <- "CTRL"

```

```

# Verify recoding
print(table(metadata$disease))

# -----
# 2. LIBRARY LOADING
# -----

suppressPackageStartupMessages({
  library(limma)
  library(edgeR)
  library(clusterProfiler)
  library(org.Hs.eg.db)
  library(ggplot2)
})

# -----
# 3. ANALYSIS CONFIGURATION
# -----

colnames(metadata) <- make.names(colnames(metadata))
output_dir <- "Results_R_Analysis"
dir.create(output_dir, showWarnings = FALSE)

cell_types <- unique(metadata$cell_type_annotation)
cat(paste("\n📋 Cell Types to analyze:", length(cell_types), "\n"))

# -----
# 4. MAIN LOOP
# -----

for (ct in cell_types) {

  cat(paste0("\n-----\n"))
  cat(paste0("Processing: ", ct, "\n"))

  # --- A. Subset ---
  samples_keep <- rownames(metadata)[metadata$cell_type_annotation == ct]

  if(length(samples_keep) < 6) {
    cat("⚠️ Skipping: Insufficient sample size (< 6 samples).\n")
    next
  }

  curr_counts <- counts[, samples_keep]

```

```

curr_meta <- metadata[samples_keep, ]

# --- B. Filter ---
keep_genes <- rowSums(cpm(curr_counts) > 1) >= (length(samples_keep) *
0.3)
curr_counts <- curr_counts[keep_genes, ]

# --- C. Norm & Model ---
dge <- DGEList(counts = curr_counts)
dge <- calcNormFactors(dge, method = "TMM")

# Use the RECODED 'disease' column
group <- factor(curr_meta$disease)

# Ensure CTRL is reference
if("CTRL" %in% levels(group)) {
  group <- relevel(group, ref = "CTRL")
} else {
  cat("⚠ Warning: No 'CTRL' group found for this cell type.
Skipping.\n")
  next
}

design <- model.matrix(~0 + group)
colnames(design) <- levels(group)

v <- voom(dge, design, plot = FALSE)
fit <- lmFit(v, design)

# Contrasts
contrasts_list <- c()
if("AD" %in% colnames(design) & "CTRL" %in% colnames(design))
contrasts_list <- c(contrasts_list, "AD - CTRL")
if("PD" %in% colnames(design) & "CTRL" %in% colnames(design))
contrasts_list <- c(contrasts_list, "PD - CTRL")

if(length(contrasts_list) == 0) {
  cat("⚠ Skipping: No valid contrasts (AD/PD vs CTRL).\n")
  next
}

cm <- makeContrasts(contrasts = contrasts_list, levels = design)
fit2 <- contrasts.fit(fit, cm)
fit2 <- eBayes(fit2)

# --- D. Results ---

for (comp_raw in contrasts_list) {
  comp_name <- gsub(" - ", "_vs_", comp_raw)

```

```

top_res <- topTable(fit2, coef = comp_raw, number = Inf, sort.by = "P")

filename_base <- file.path(output_dir, paste0(make.names(ct), "_",
comp_name))
write.csv(top_res, paste0(filename_base, ".csv"))

n_sig <- sum(top_res$adj.P.Val < 0.05 & abs(top_res$logFC) > 0.5)
cat(paste("    -> comparison", comp_name, ":", n_sig, "DEGs found.\n"))

# Volcano Plot
try({
  df_plot <- top_res
  df_plot$Significance <- "NS"
  df_plot$Significance[df_plot$adj.P.Val < 0.05 & df_plot$logFC > 0.5]
<- "UP"
  df_plot$Significance[df_plot$adj.P.Val < 0.05 & df_plot$logFC < -0.5]
<- "DOWN"

  p_vol <- ggplot(df_plot, aes(x = logFC, y = -log10(adj.P.Val), color
= Significance)) +
    geom_point(alpha = 0.6, size = 1.5) +
    scale_color_manual(values = c("DOWN" = "blue", "NS" = "grey", "UP"
= "red")) +
    theme_minimal() +
    geom_vline(xintercept = c(-0.5, 0.5), linetype = "dashed") +
    geom_hline(yintercept = -log10(0.05), linetype = "dashed") +
    labs(title = paste(ct, comp_name),
         subtitle = paste("DEGs:", n_sig),
         x = "Log2 Fold Change", y = "-Log10 FDR") +
    theme(legend.position = "top")

  ggsave(paste0(filename_base, "_Volcano.pdf"), p_vol, width = 6,
height = 5)
  }, silent = TRUE)

# Enrichment
sig_genes <- rownames(top_res)[top_res$adj.P.Val < 0.05 &
abs(top_res$logFC) > 0.5]

if (length(sig_genes) >= 5) {
  cat("    -> Running GO Enrichment...\n")
  try({
    ego <- enrichGO(gene = sig_genes, OrgDb = org.Hs.eg.db, keyType =
"SYMBOL",
                    ont = "BP", pAdjustMethod = "BH", qvalueCutoff =
0.05)

    if (!is.null(ego) && nrow(ego) > 0) {

```

```

        p_dot <- dotplot(ego, showCategory=15) + ggtitle(paste("GO:", ct,
comp_name))
        ggsave(paste0(filename_base, "_GO_Dotplot.pdf"), p_dot, width =
10, height = 7)
    }
    }, silent = TRUE)
}
}
}

cat("\n=====
==\n")
cat("✅ PIPELINE COMPLETE. Results saved in 'Results_R_Analysis'
folder.\n")
cat("=====
\n")

```

1. Setup and Configuration

In [1]:

```

import os
import re
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

#
=====
==
# PATH CONFIGURATION
#
=====
==

# Root path (Current working directory)
BASE_PATH = os.getcwd()

# Input directory (R pipeline output) and Output directory (Final Figures)
INPUT_DIR = os.path.join(BASE_PATH, "Results_R_Analysis")
OUTPUT_DIR = os.path.join(BASE_PATH, "Figures_Finales")

# Create output subdirectories structure
VOLCANO_DIR = os.path.join(OUTPUT_DIR, "volcano")
HEATMAP_DIR = os.path.join(OUTPUT_DIR, "heatmaps")
BARPLOT_DIR = os.path.join(OUTPUT_DIR, "barplots")

for d in [VOLCANO_DIR, HEATMAP_DIR, BARPLOT_DIR]:

```

```

os.makedirs(d, exist_ok=True)

SUMMARY_CSV = os.path.join(OUTPUT_DIR, "summary_DEGs.csv")

# Plotting Style (Seaborn Whitegrid for publication quality)
sns.set_theme(style="whitegrid")
plt.rcParams["figure.dpi"] = 100          # Display DPI
plt.rcParams["savefig.dpi"] = 300        # Export DPI (High Res)
plt.rcParams["font.size"] = 10

print(f"📁 Input Directory: {INPUT_DIR}")
print(f"📁 Output Directory: {OUTPUT_DIR}")
📁 Input Directory: c:\Z\AIDA_transcriptomics_project\transcriptomics-
code\Results_R_Analysis
📁 Output Directory: c:\Z\AIDA_transcriptomics_project\transcriptomics-
code\Figures_Finales

```

2. Utility Functions

Helper functions to parse filenames and handle data columns safely.

In [2]:

```

def find_column(df, candidates, required=True):
    """
    Returns the name of the first column found in the dataframe from a list
    of candidates.
    Handles case-insensitivity.
    """
    for col in candidates:
        if col in df.columns:
            return col
        for real_col in df.columns:
            if real_col.lower() == col.lower():
                return real_col
    if required:
        raise ValueError(f"None of the candidate columns {candidates} were
found in the CSV.")
    return None

def safe_log10(pvalues):
    """
    Returns -log10(pvalue), handling zeros and NaNs safely.
    """
    pvalues = pvalues.astype(float)
    pvalues = pvalues.replace(0, np.nan)
    with np.errstate(divide="ignore"):
        res = -np.log10(pvalues)

```

```

        return res

def parse_name_from_filename(fname):
    """
    Extracts (CellType, Contrast) from the filename.
    Expected format: 'Microglia_AD_vs_CTRL_results.csv'
    """
    base = os.path.basename(fname)
    base = re.sub(r"\.csv$", "", base)

    # Pattern: Microglia_AD_vs_CTRL
    m = re.match(r"(.+?)_([A-Za-z0-9]+_vs_[A-Za-z0-9]+).*", base)
    if m:
        return m.group(1), m.group(2)
    return base, None

```

3. Plotting Functions

Core logic for generating the three types of required figures: Volcano, Heatmap, and Barplot.

```

In [3]:
def plot_volcano(df, gene_col, lfc_col, p_col, padj_col, title, out_path,
top_n_labels=10):
    """
    Generates a 'Smart' Volcano Plot with trinary zoning:
    1. GRAY: Non-significant
    2. ORANGE: Trend (Raw P < 0.01 but FDR >= 0.05) - Crucial for low-power
datasets
    3. RED: Significant (FDR < 0.05)
    """
    df = df.copy()
    df = df.replace([np.inf, -np.inf], np.nan).dropna(subset=[lfc_col,
p_col])
    if df.empty: return

    df["minus_log10_p"] = safe_log10(df[p_col])

    # --- Define Categories ---
    df["category"] = "non_signif"

    # 1. Biological Trend (Orange)
    df.loc[df[p_col] < 0.01, "category"] = "trend"

    # 2. Robust Signal (Red) - Overwrites trend if FDR is good
    if padj_col in df.columns:
        df.loc[df[padj_col] < 0.05, "category"] = "signif"

```



```

palette = {"non_signif": "lightgray", "trend": "orange", "signif":
"red"}

# Plotting
plt.figure(figsize=(6, 5))
ax = sns.scatterplot(
    data=df, x=lfc_col, y="minus_log10_p", hue="category",
    palette=palette, edgecolor=None, alpha=0.7, s=25
)

# Threshold lines
ax.axvline(0, color="black", linewidth=0.8, linestyle="--")
ax.axhline(-np.log10(0.01), color="orange", linewidth=0.8,
linestyle=":", alpha=0.5)

ax.set_xlabel("log2(Fold Change)")
ax.set_ylabel("-log10(Raw P-value)")
ax.set_title(title, fontsize=11, fontweight='bold')
ax.legend(title="", loc="upper right")

# --- Annotation Strategy ---
# We annotate genes with the lowest RAW p-value (strongest signal)
# regardless of FDR significance, to facilitate biological discussion.
df_sorted = df.sort_values(by=p_col, ascending=True).head(top_n_labels)

for _, row in df_sorted.iterrows():
    gene = str(row[gene_col])
    if row["category"] == "non_signif": continue # Do not annotate
noise

    ax.text(row[lfc_col], row["minus_log10_p"], gene,
            fontsize=8, ha="right", va="bottom", fontweight='bold')

plt.tight_layout()
plt.savefig(out_path, bbox_inches="tight")
plt.close()
print(f"    --> Volcano saved: {os.path.basename(out_path)}")

def plot_heatmap_top_trends(df, gene_col, lfc_col, p_col, title, out_path,
top_n=20):
    """
    Generates a heatmap of the 'Top Trends': showing the top 20 genes
    (sorted by Raw P-value)
    to identify potential signatures even without FDR significance.
    """
    df = df.copy().dropna(subset=[lfc_col, p_col])
    if df.empty: return

    # Sort by Raw P-value

```

```

df_sorted = df.sort_values(by=p_col, ascending=True).head(top_n)

mat = df_sorted[[lfc_col]]
mat.index = df_sorted[gene_col]

plt.figure(figsize=(4, max(4, len(mat) * 0.25)))
sns.heatmap(mat, cmap="coolwarm", center=0, annot=True, fmt=".2f",
            cbar_kws={"label": "log2FC"})
plt.title(title, fontsize=10)
plt.xlabel("")
plt.tight_layout()
plt.savefig(out_path, bbox_inches="tight")
plt.close()
print(f"    --> Heatmap saved: {os.path.basename(out_path)}")

def plot_barplot_top_genes(df, gene_col, lfc_col, title, out_path,
top_n=15):
    """
    Horizontal Barplot of the top N genes with the highest absolute logFC.
    """
    df = df.copy().dropna(subset=[lfc_col])
    if df.empty: return

    df["abs_lfc"] = df[lfc_col].abs()
    df_sorted = df.sort_values("abs_lfc",
ascending=False).head(top_n).iloc[::-1]

    colors = ["red" if x > 0 else "royalblue" for x in df_sorted[lfc_col]]

    plt.figure(figsize=(6, max(4, len(df_sorted) * 0.3)))
    plt.barh(df_sorted[gene_col], df_sorted[lfc_col], color=colors)
    plt.axvline(0, color="black", linewidth=0.8)
    plt.title(title)
    plt.xlabel("log2 (Fold Change)")
    plt.tight_layout()
    plt.savefig(out_path, bbox_inches="tight")
    plt.close()
    print(f"    --> Barplot saved: {os.path.basename(out_path)}")

```

4. Main Execution Pipeline

Iterates through all CSV files in the input directory and generates the corresponding figures.

In [4]:

```

summary_rows = []
csv_files = [f for f in os.listdir(INPUT_DIR) if
f.lower().endswith(".csv")]

```

```

if not csv_files:
    print(f"❌ ERROR: No CSV files found in {INPUT_DIR}")
else:
    print(f"🚀 Processing {len(csv_files)} files...")
    print("-" * 60)

    for fname in csv_files:
        fpath = os.path.join(INPUT_DIR, fname)

        try:
            df = pd.read_csv(fpath)
        except Exception as e:
            print(f"      [ERROR] Could not read file: {fname}")
            continue

        # --- Column Detection ---
        try:
            gene_col = find_column(df, ["gene", "Gene", "GENE", "symbol",
"Row.names", "Unnamed: 0"])
            lfc_col = find_column(df, ["log2FoldChange", "logFC",
"log2FC"])
            p_col = find_column(df, ["pvalue", "p.value", "P.Value",
"PValue"])
            padj_col = find_column(df, ["padj", "adj.P.Val", "FDR",
"qvalue"], required=False)
        except ValueError as e:
            print(f"      [SKIP] Missing required columns in {fname}")
            continue

        cell_type, contrast = parse_name_from_filename(fname)
        title_base = f"{cell_type}" + (f" - {contrast}" if contrast else
"")

        # --- Summary Calculations ---
        total_genes = len(df)
        n_trends = int(df[p_col].lt(0.01).sum()) # Trends (Orange)
        n_signif = 0
        if padj_col:
            n_signif = int(df[padj_col].lt(0.05).sum()) # Significant (Red)

        # Top 3 Genes (Names)
        top3 =
df.sort_values(by=p_col).head(3)[gene_col].astype(str).tolist()
        top3_str = ", ".join(top3)

        summary_rows.append({
            "File": fname,
            "Cell Type": cell_type,

```

```

        "Contrast": contrast,
        "Total Genes": total_genes,
        "Trends (P<0.01)": n_trends,
        "Significant (FDR<0.05)": n_signif,
        "Top Genes": top3_str
    })

    # --- Generate Figures ---
    file_suffix = f"{cell_type}" + (f"_{contrast}" if contrast else "")

    # 1. Volcano
    plot_volcano(df, gene_col, lfc_col, p_col, padj_col,
                 f"{title_base} (Volcano)",
                 os.path.join(VOLCANO_DIR,
f"Volcano_{file_suffix}.png"))

    # 2. Heatmap
    plot_heatmap_top_trends(df, gene_col, lfc_col, p_col,
                            f"{title_base} (Top Trends)",
                            os.path.join(HEATMAP_DIR,
f"Heatmap_{file_suffix}.png"))

    # 3. Barplot
    plot_barplot_top_genes(df, gene_col, lfc_col,
                           f"{title_base} (Top |logFC|)",
                           os.path.join(BARPLOT_DIR,
f"Barplot_{file_suffix}.png"))
    print("-" * 40)

    # --- Export Summary ---
    if summary_rows:
        summary_df = pd.DataFrame(summary_rows)
        summary_df.to_csv(SUMMARY_CSV, index=False)
        print(f"\n✅ Summary Table generated: {SUMMARY_CSV}")
        display(summary_df)
    else:
        print("⚠️ No results processed.")

```

🚀 Processing 13 files...

```

-----
--> Volcano saved: Volcano_Astrocyte.Homeostatic_AD_vs_CTRL.png
--> Heatmap saved: Heatmap_Astrocyte.Homeostatic_AD_vs_CTRL.png
--> Barplot saved: Barplot_Astrocyte.Homeostatic_AD_vs_CTRL.png
-----
--> Volcano saved: Volcano_Astrocyte.Homeostatic_PD_vs_CTRL.png
--> Heatmap saved: Heatmap_Astrocyte.Homeostatic_PD_vs_CTRL.png
--> Barplot saved: Barplot_Astrocyte.Homeostatic_PD_vs_CTRL.png
-----
--> Volcano saved: Volcano_Astrocyte.Reactive_AD_vs_CTRL.png
--> Heatmap saved: Heatmap_Astrocyte.Reactive_AD_vs_CTRL.png

```

```

--> Barplot saved: Barplot_Astrocyte.Reactive_AD_vs_CTRL.png
-----
--> Volcano saved: Volcano_Excitatory.neuron_AD_vs_CTRL.png
--> Heatmap saved: Heatmap_Excitatory.neuron_AD_vs_CTRL.png
--> Barplot saved: Barplot_Excitatory.neuron_AD_vs_CTRL.png
-----
--> Volcano saved: Volcano_Excitatory.neuron_PD_vs_CTRL.png
--> Heatmap saved: Heatmap_Excitatory.neuron_PD_vs_CTRL.png
--> Barplot saved: Barplot_Excitatory.neuron_PD_vs_CTRL.png
-----
--> Volcano saved: Volcano_Inhibitory.neuron_AD_vs_CTRL.png
--> Heatmap saved: Heatmap_Inhibitory.neuron_AD_vs_CTRL.png
--> Barplot saved: Barplot_Inhibitory.neuron_AD_vs_CTRL.png
-----
--> Volcano saved: Volcano_Inhibitory.neuron_PD_vs_CTRL.png
--> Heatmap saved: Heatmap_Inhibitory.neuron_PD_vs_CTRL.png
--> Barplot saved: Barplot_Inhibitory.neuron_PD_vs_CTRL.png
-----
--> Volcano saved: Volcano_Microglia_AD_vs_CTRL.png
--> Heatmap saved: Heatmap_Microglia_AD_vs_CTRL.png
--> Barplot saved: Barplot_Microglia_AD_vs_CTRL.png
-----
--> Volcano saved: Volcano_Microglia_PD_vs_CTRL.png
--> Heatmap saved: Heatmap_Microglia_PD_vs_CTRL.png
--> Barplot saved: Barplot_Microglia_PD_vs_CTRL.png
-----
--> Volcano saved: Volcano_Oligodendrocyte.OPC_AD_vs_CTRL.png
--> Heatmap saved: Heatmap_Oligodendrocyte.OPC_AD_vs_CTRL.png
--> Barplot saved: Barplot_Oligodendrocyte.OPC_AD_vs_CTRL.png
-----
--> Volcano saved: Volcano_Oligodendrocyte.OPC_PD_vs_CTRL.png
--> Heatmap saved: Heatmap_Oligodendrocyte.OPC_PD_vs_CTRL.png
--> Barplot saved: Barplot_Oligodendrocyte.OPC_PD_vs_CTRL.png
-----
--> Volcano saved: Volcano_Support.Vascular.Immune_AD_vs_CTRL.png
--> Heatmap saved: Heatmap_Support.Vascular.Immune_AD_vs_CTRL.png
--> Barplot saved: Barplot_Support.Vascular.Immune_AD_vs_CTRL.png
-----
--> Volcano saved: Volcano_Support.Vascular.Immune_PD_vs_CTRL.png
--> Heatmap saved: Heatmap_Support.Vascular.Immune_PD_vs_CTRL.png
--> Barplot saved: Barplot_Support.Vascular.Immune_PD_vs_CTRL.png
-----

```

 Summary Table generated:
c:\Z\AIDA_transcriptomics_project\transcriptomics-
code\Figures_Finales\summary_DEGs.csv

	File	Cell Type	Contrast	Total Genes	Trend (P<0.05)
0	Astrocyte.Homeostatic_AD_vs_CTRL.csv	Astrocyte.Homeostatic	AD_vs_CTRL	1578	4
1	Astrocyte.Homeostatic_PD_vs_CTRL.csv	Astrocyte.Homeostatic	PD_vs_CTRL	1578	28
2	Astrocyte.Reactive_AD_vs_CTRL.csv	Astrocyte.Reactive	AD_vs_CTRL	1464	9
3	Excitatory.neuron_AD_vs_CTRL.csv	Excitatory.neuron	AD_vs_CTRL	1607	10
4	Excitatory.neuron_PD_vs_CTRL.csv	Excitatory.neuron	PD_vs_CTRL	1607	12
5	Inhibitory.neuron_AD_vs_CTRL.csv	Inhibitory.neuron	AD_vs_CTRL	1764	6
6	Inhibitory.neuron_PD_vs_CTRL.csv	Inhibitory.neuron	PD_vs_CTRL	1764	1
7	Microglia_AD_vs_CTRL.csv	Microglia	AD_vs_CTRL	1504	26
8	Microglia_PD_vs_CTRL.csv	Microglia	PD_vs_CTRL	1504	2
9	Oligodendrocyte.OPC_AD_vs_CTRL.csv	Oligodendrocyte.OPC	AD_vs_CTRL	1602	5
10	Oligodendrocyte.OPC_PD_vs_CTRL.csv	Oligodendrocyte.OPC	PD_vs_CTRL	1602	7
11	Support.Vascular.Immune_AD_vs_CTRL.csv	Support.Vascular.Immune	AD_vs_CTRL	1555	5
12	Support.Vascular.Immune_PD_vs_CTRL.csv	Support.Vascular.Immune	PD_vs_CTRL	1555	10

5. Conclusion

Figures have been generated in the `Figures_Finales` directory. You can now integrate them into the PDF report.